

The Foundations: Logic and Proofs

Chapter 3: Algorithms

With Question/Answer Animations

19. Express each of these statements using mathematical and logical operators, predicates, and quantifiers, where the domain consists of all integers.

- a) The sum of two negative integers is negative.
- b) The difference of two positive integers is not necessarily positive.

19:

$\forall$ $\rightarrow$ $\wedge$

正确 $\neg (\forall x \forall y ((x > 0) \wedge (y > 0) \rightarrow x - y > 0))$

$\equiv \exists x \neg \forall y ((x > 0) \wedge (y > 0) \rightarrow x - y > 0)$

$\equiv \exists x \exists y ((x > 0) \wedge (y > 0) \wedge (x - y \leq 0))$ $\begin{matrix} p \rightarrow q \\ \equiv \neg p \vee q \end{matrix}$

正确 $\equiv \exists x \exists y ((x > 0) \wedge (y > 0) \wedge (x - y \leq 0))$

不可以 $\exists x \exists y ((x > 0) \wedge (y > 0) \rightarrow x - y \leq 0)$

The Growth of Functions

Section 3.2

The Growth of Functions

In both computer science and in mathematics, there are many times when we care about how fast a function grows.

In computer science, we want to understand how quickly an algorithm can solve a problem as the size of the input grows.

- We can compare the efficiency of two different algorithms for solving the same problem.
- We can also determine whether it is practical to use a particular algorithm as the input grows.
- We'll study these questions in Section 3.3.

3.2.2 Big-O Notation₁

Definition: Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that $f(x)$ is $O(g(x))$ if there are constants C and k such that

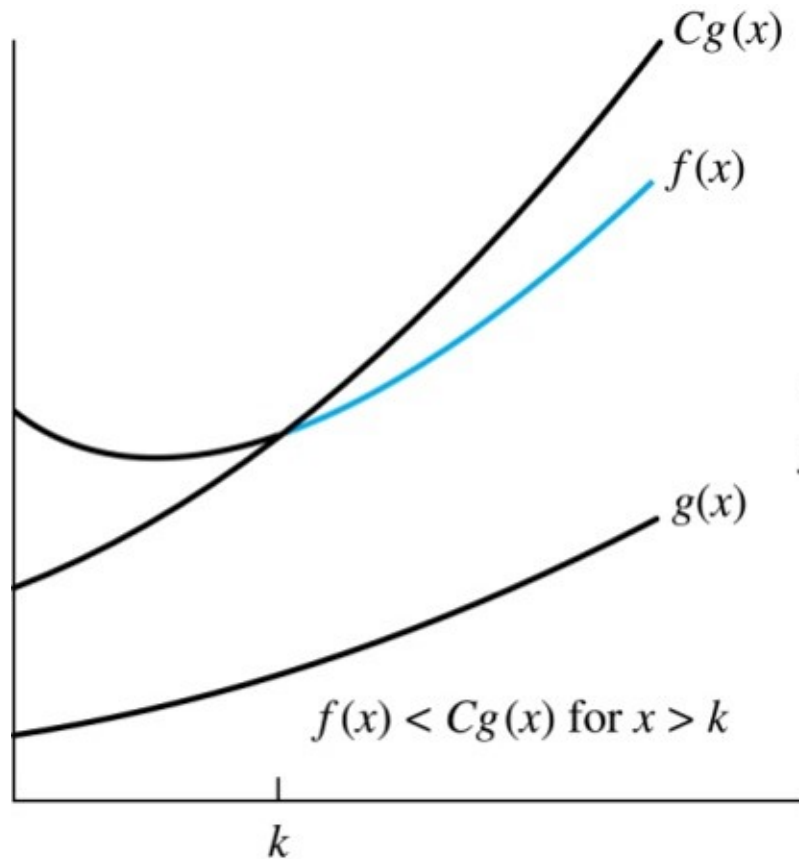
$$|f(x)| \leq C |g(x)|$$

whenever $x > k$.

This is read as “ $f(x)$ is big- O of $g(x)$ ” or “ g asymptotically dominates f .”

The constants C and k are called *witnesses* (证人) to the relationship $f(x)$ is $O(g(x))$. Only one pair of witnesses is needed.

Illustration of Big- O Notation₁



$$f(x) \text{ is } O(g(x))$$

The part of the graph of $f(x)$ that satisfies $f(x) < Cg(x)$ is shown in color.

[Jump to long description](#)

Using the Definition of Big-O Notation₁

Example: Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$.

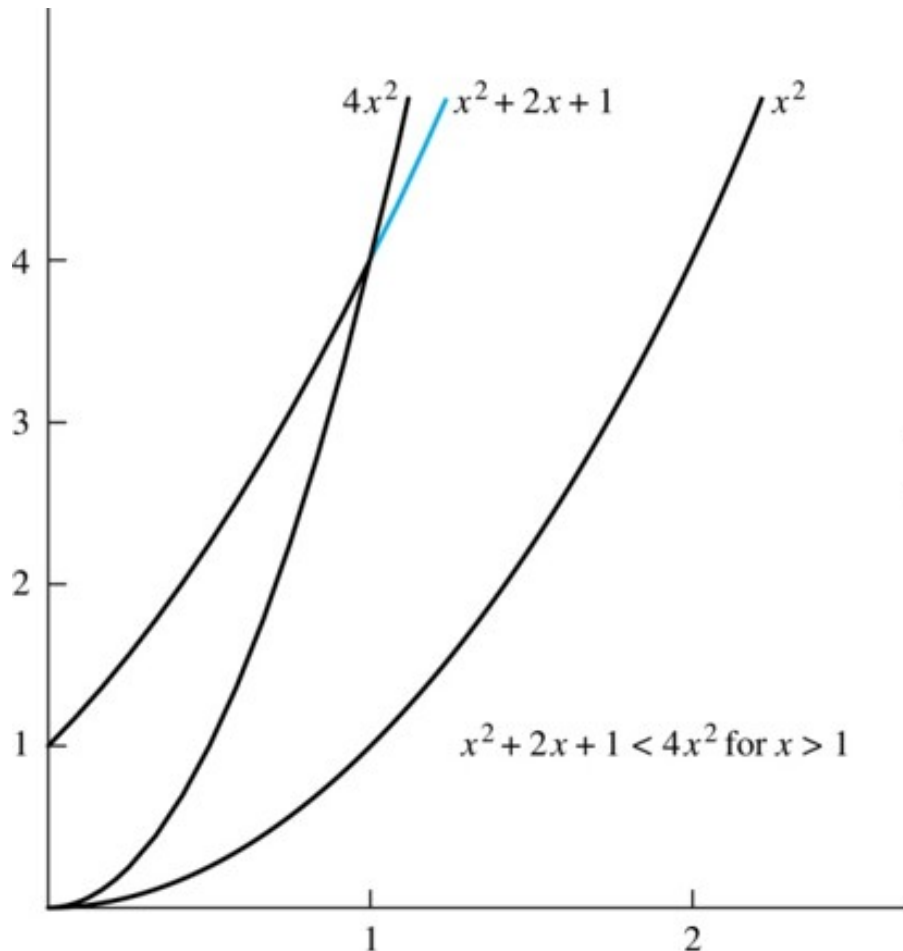
Solution: Since when $x > 1$, $x < x^2$ and $1 < x^2$

$$0 \leq x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2$$

- We can take $C = 4$ and $k = 1$ as witnesses to show that $f(x)$ is $O(x^2)$.
- Alternatively, when $x > 2$, we have $2x \leq x^2$ and $1 < x^2$. Hence, $0 \leq x^2 + 2x + 1 \leq x^2 + x^2 + x^2 = 3x^2$ when $x > 2$.
- Can take $C = 3$ and $k = 2$ as witnesses instead.

Illustration of Big-O Notation₂

$$f(x) = x^2 + 2x + 1 \text{ is } O(x^2)$$



The part of the graph of $f(x) = x^2 + 2x + 1$ that satisfies $f(x) < 4x^2$ is shown in blue.

$$x^2 + 2x + 1 < 4x^2 \text{ for } x > 1$$

Using the Definition of Big-O Notation₂

Example: Show that $7x^2$ is $O(x^3)$.

Solution: When $x > 7$, $7x^2 < x^3$. Take $C = 1$ and $k = 7$ as witnesses to establish that $7x^2$ is $O(x^3)$.

(Would $C = 7$ and $k = 1$ work?)

Example: Show that n^2 is not $O(n)$.

Solution: Suppose there are constants C and k for which $n^2 \leq Cn$, whenever $n > k$.

We can divide both sides of $n^2 \leq Cn$ by n , then $n \leq C$ must hold for all $n > k$. A contradiction!

3.2.3 Big-O Estimates for Some Important Functions

Big-O Estimates for Polynomials

Example: Let $f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$

where $a_0, a_1, \dots, a_n$ are real numbers with $a_n \neq 0$. Then $f(x)$ is $O(x^n)$.

Proof:

$$\begin{aligned} |f(x)| &= |a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x^1 + a_0| && \text{Assuming } x > 1 \\ &\leq |a_n| x^n + |a_{n-1}| x^{n-1} + \cdots + |a_1| x^1 + |a_0| && \text{Using triangle inequality.} \\ &= x^n \left(|a_n| + |a_{n-1}|/x + \cdots + |a_1|/x^{n-1} + |a_0|/x^n \right) \\ &\leq x^n \left(|a_n| + |a_{n-1}| + \cdots + |a_1| + |a_0| \right) \end{aligned}$$

Take $C = |a_n| + |a_{n-1}| + \cdots + |a_0|$ and $k = 1$. Then $f(x)$ is $O(x^n)$.

The leading term $a_n x^n$ of a polynomial dominates its growth.

Big- O Estimates for some Important Functions₁

Example: Use big- O notation to estimate the sum of the first n positive integers.

Solution: $1 + 2 + \cdots + n \leq n + n + \cdots + n = n^2$

$1 + 2 + \cdots + n$ is $O(n^2)$ taking $C = 1$ and $k = 1$.

Example: Use big- O notation to estimate the factorial function $f(n) = n! = 1 \times 2 \times \cdots \times n$.

Solution:

$$n! = 1 \times 2 \times \cdots \times n \leq n \times n \times \cdots \times n = n^n$$

$n!$ is $O(n^n)$ taking $C = 1$ and $k = 1$.

Big- O Estimates for some Important Functions₂

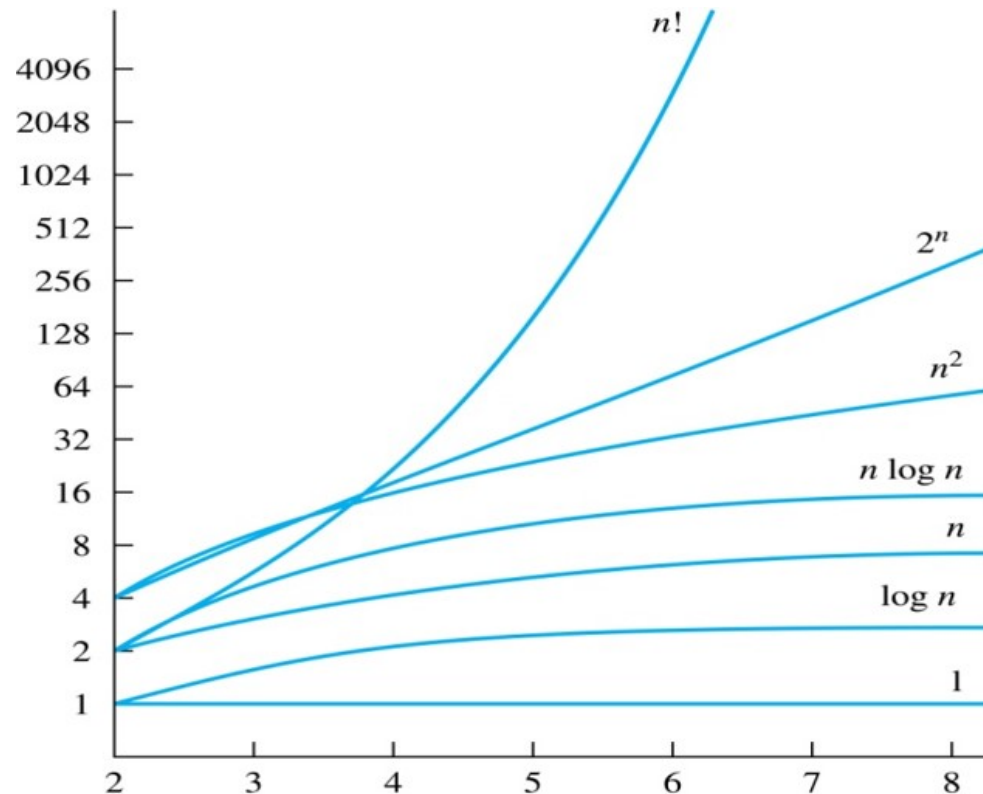
Example: Use big- O notation to estimate $\log n!$

Solution: Given that $n! \leq n^n$ (previous slide)

then $\log(n!) \leq n \cdot \log(n)$.

Hence, $\log(n!)$ is $O(n \cdot \log(n))$ taking $C = 1$ and $k = 1$.

Display of Growth of Functions



Note the difference in behavior of functions as n gets larger

[Jump to long description](#)

大O符号（英语：Big O notation），又称为渐进符号，是用于描述函数渐近行为的数学符号。更确切地说，它是用另一个（通常更简单的）函数来描述一个函数数量级的渐近上界。在数学中，它一般用来刻画被截断的无穷级数尤其是渐近级数的剩余项；在计算机科学中，它在分析算法复杂性的方面非常有用。

- 1、 $O(1)$ 为常数级的时间复杂度，算法是十分好。
- 2、 $O(\log n)$ 为对数级的时间复杂度，算法也不错。
- 3、 $O(n)$ 为线性级的时间复杂度，算法也还行。
- 4、 $O(n \log n)$ 线性对数级的时间复杂度，算法也还可以。
- 5、 $O(n^2)$ 二次方级的时间复杂度，算法有点差。
- 6、 $O(n^3)$ 三次方级的时间复杂度，算法差。
- 7、 $O(2^n)$ 指数级的时间复杂度，算法很差。
- 8、 $O(n!)$ 阶乘级的时间复杂度，算法极差。

大O准确来说，使用另一个函数来描述一个函数数量级的**渐进上界

3.2.5 Big-Omega and Big-Theta Notation

Big-Omega Notation₁

Definition: Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that $f(x)$ is $\Omega(g(x))$

if there are constants C and k such that

$$|f(x)| \geq C |g(x)| \quad \text{when } x > k.$$

Ω is the upper case version of the lower case Greek letter ω .s

We say that “ $f(x)$ is big-Omega of $g(x)$.”

Big- O gives an upper bound on the growth of a function, while Big-Omega gives a lower bound. Big-Omega tells us that a function grows at least as fast as another.

$f(x)$ is $\Omega(g(x))$ if and only if $g(x)$ is $O(f(x))$. This follows from the definitions.

大 Ω 符号的定义与大 O 符号的定义类似，但主要区别是，大 O 符号表示函数在增长到一定程度时总小于一个特定函数的常数倍，大 Ω 符号则表示总大于。大 Ω 符号主要是用来描述一个函数数量级的渐进下

界**。

Big-Omega Notation₂

Example: Show that $f(x) = 8x^3 + 5x^2 + 7$ is $\Omega(g(x))$ where $g(x) = x^3$.

Solution: $f(x) = 8x^3 + 5x^2 + 7 \geq 8x^3$ for all positive real numbers x .

Big-Theta Notation₁

Θ is the upper case version of the lower case Greek letter θ .

Definition: Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. The function

$f(x)$ is $\Theta(g(x))$ if $f(x)$ is $O(g(x))$ and $f(x)$ is $\Omega(g(x))$.

We say that “ f is big-Theta of $g(x)$ ” and also that “ $f(x)$ is of order $g(x)$ ” and also that “ $f(x)$ and $g(x)$ are of the same order (同阶).”

$f(x)$ is $\Theta(g(x))$ if and only if there are positive real numbers

C_1 and C_2 and a positive real number k such that

$$C_1|g(x)| \leq |f(x)| \leq C_2|g(x)| \quad \text{where } x > k.$$

This follows from the definitions of big- O and big- Ω .

When $f(x)$ is $\Theta(g(x))$ it must also be the case that $g(x)$ is $\Theta(f(x))$.

O 表示上界，小于等于的意思。

Ω 表示下界，大于等于的意思。

$f(x)=\Theta(g(x))$ ，这个才是同阶量

Big-Theta Notation₃

Example: Show that $f(x) = 3x^2 + 8x \log x$ is $\Theta(x^2)$.

Solution:

$$3x^2 + 8x \log x \leq 11x^2 \text{ for } x > 1,$$

$$\text{since } 0 \leq 8x \log x \leq 8x^2.$$

$$\text{Hence, } 3x^2 + 8x \log x \text{ is } O(x^2).$$

$$x^2 \text{ is clearly } O(3x^2 + 8x \log x)$$

$$\text{Hence, } 3x^2 + 8x \log x \text{ is } \Theta(x^2).$$

Big-Theta Estimates for Polynomials

Theorem: Let $f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$

where $a_0, a_1, \dots, a_n$ are real numbers with $a_n \neq 0$.

Then $f(x)$ is of order x^n (or $\Theta(x^n)$).

Example:

The polynomial $f(x) = 8x^5 + 5x^2 + 10$ is of order x^5 (or $\Theta(x^5)$).

The polynomial $f(x) = 8x^{199} + 7x^{100} + x^{99} + 5x^2 + 25$ is of order x^{199} (or $\Theta(x^{199})$).

Section Summarys

Big-O Notation $f(x)$ is $O(g(x))$ $|f(x)| \leq C |g(x)|$ $x > k$

Big-O Estimates for Important Functions

Big-Omega and Big-Theta Notation

$f(x)$ is $\Omega(g(x))$ $|f(x)| \geq C |g(x)|$

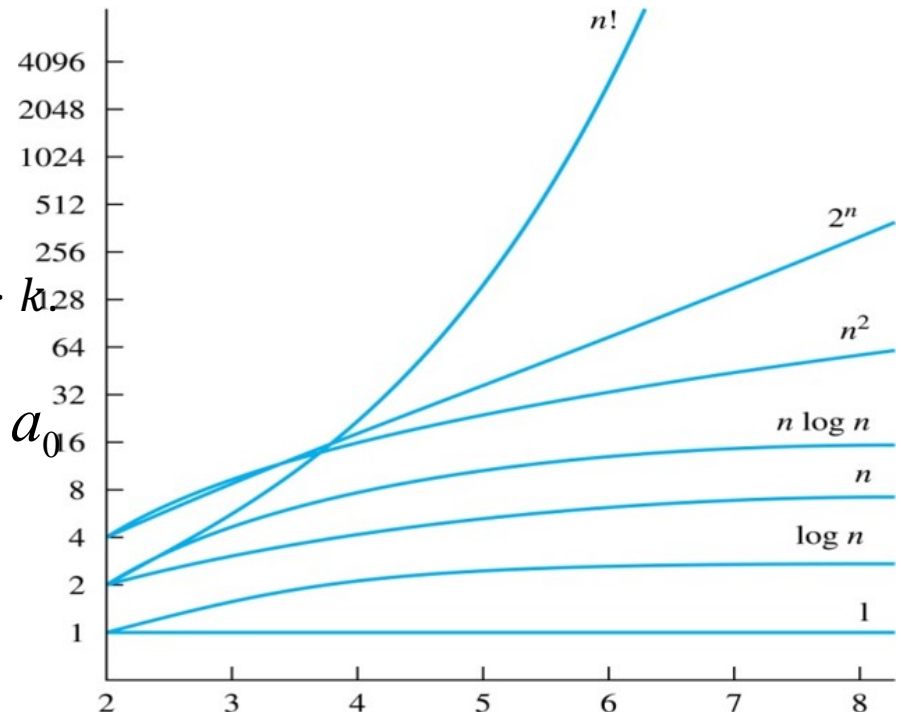
$f(x)$ is $\Theta(g(x))$

$C_1 |g(x)| \leq |f(x)| \leq C_2 |g(x)|$ where $x > k$

$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$

$f(x)$ is $O(x^n)$.

$f(x)$ is of order x^n (or $\Theta(x^n)$).



Exercise

1. Determine whether each of these functions is $O(x)$.

a) $f(x) = 10$

b) $f(x) = 3x + 7$

c) $f(x) = x^2 + x + 1$

d) $f(x) = 5 \log x$

28. For each function in Exercise 1, determine whether that function is $\Omega(x)$ and whether it is $\Theta(x)$.

Complexity of Algorithms

Section 3.3

The Complexity of Algorithms₁

Given an algorithm, how efficient is this algorithm for solving a problem given input of a particular size? To answer this question, we ask:

- How much time does this algorithm use to solve a problem?
- How much computer memory does this algorithm use to solve a problem?

When we analyze the time the algorithm uses to solve the problem given input of a particular size, we are studying the *time complexity* of the algorithm.

When we analyze the computer memory the algorithm uses to solve the problem given input of a particular size, we are studying the *space complexity* of the algorithm.

3.3.2 Time Complexity

To analyze the **time complexity**(时间复杂度) of algorithms, we determine the **number of operations**, such as **comparisons** and **arithmetic operations** (addition, multiplication, etc.). We can estimate the time a computer may actually use to solve a problem using the amount of time required to do basic operations.

We ignore minor details, such as the “house keeping” aspects of the algorithm.

We will focus on the **worst-case time complexity** of an algorithm. This provides an upper bound on the number of operations an algorithm uses to solve a problem with input of a particular size.

It is usually much more difficult to determine the **average case time complexity** of an algorithm. This is the average number of operations an algorithm uses to solve a problem over all inputs of a particular size.

Searching Problems

Definition: The general *searching problem* is to locate an element x in the list of distinct elements $a_1, a_2, \dots, a_n$, or determine that it is not in the list.

- The solution to a searching problem is the location of the term in the list that equals x (that is, i is the solution if $x = a_i$ or 0 if x is not in the list.
- For example, a library might want to check to see if a patron is on a list of those with overdue books before allowing him/her to checkout another book.
- We will study two different searching algorithms: **linear search** and **binary search**.

Linear Search Algorithm

The linear search algorithm locates an item in a list by examining elements in the sequence one at a time, starting at the beginning.

- First compare x with a_1 . If they are equal, return the position 1.
- If not, try a_2 . If $x = a_2$, return the position 2.
- Keep going, and if no match is found when the entire list is scanned, return 0.

```
procedure linear search( $x$ :integer,  
                         $a_1, a_2, \dots, a_n$ : distinct integers)  
 $i := 1$   
while ( $i \leq n$  and  $x \neq a_i$ )  
     $i := i + 1$   
if  $i \leq n$  then  $location := i$   
else  $location := 0$   
return  $location$ { $location$  is the subscript of the term that  
equals  $x$ , or is 0 if  $x$  is not found}
```

语法

C 语言中 **while** 循环的语法：

```
while(condition)
{
    statement(s);
}
```

在这里，**statement(s)** 可以是一个单独的语句，也可以是几个语句组成的代码块。

condition 可以是任意的表达式，当为任意非零值时都为 true。当条件为 true 时执行循环。当条件为 false 时，退出循环，程序流将继续执行紧接着循环的下一条语句。

Ex2: Worst-Case Complexity of Linear Search(线性搜寻)

Example: Determine the time complexity of the linear search algorithm.

```
procedure linear search( $x$ :integer,  
                         $a_1, a_2, \dots, a_n$ : distinct integers)  
 $i := 1$   
while ( $i \leq n$  and  $x \neq a_i$ )  
     $i := i + 1$   
if  $i \leq n$  then  $location := i$   
else  $location := 0$   
return  $location$ { $location$  is the subscript of the term that equals  $x$ , or is 0  
if  $x$  is not found}
```

Solution: Count the number of comparisons.

- At each step two comparisons are made; $i \leq n$ and $x \neq a_i$.
- To end the loop, one comparison $i \leq n$ is made.
- After the loop, one more $i \leq n$ comparison is made.

If $x = a_i$, $2i + 1$ comparisons are used. If x is not on the list, $2n$ comparisons are used to determine that x is not a_i , for $i = 1, 2, \dots, n$, an additional comparison is used to exit the loop, and one comparison is made outside the loop. So, in the worst case $2n + 2$ comparisons are made. Hence, the complexity is $\Theta(n)$.

Ex4: Average-Case Complexity of Linear Search

Example: Describe the average case performance of the linear search algorithm. (Although usually it is very difficult to determine average-case complexity, it is easy for linear search.)

Solution: Assume the element is in the list and that the possible positions are equally likely. By the argument on the previous slide, if $x = a_i$, the number of comparisons is $2i + 1$.

$$\frac{3 + 5 + 7 + \dots + (2n + 1)}{n} = \frac{2(1 + 2 + 3 + \dots + n) + n}{n} = \frac{2 \left[\frac{n(n+1)}{2} \right] + n}{n} + 1 = n + 2$$

Hence, the average-case complexity of linear search is $\Theta(n)$.

Binary Search₁

Assume the input is a list of items in **increasing order**.

The algorithm begins by comparing the element to be found with the middle element.

- If the middle element is lower, the search proceeds with the upper half of the list.
- If it is not lower, the search proceeds with the lower half of the list (through(通过) the middle position).

Repeat this process until we have a list of size 1.

- If the element we are looking for is equal to the element in the list, the position is returned.
- Otherwise, 0 is returned to indicate that the element was not found.

In Section 3.3, we show that the binary search algorithm is much more efficient than linear search.

Binary Search₃

Example: The steps taken by a binary search for 19 in the list:

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

1. The list has 16 elements, so the midpoint is 8. The value in the 8th position is 10. Since $19 > 10$, further search is restricted to positions 9 through 16.

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

$i=1, j=16, m=8, 19 > a_8=10, i=m+1=9$

2. The midpoint of the list (positions 9 through 16) is now the 12th position with a value of 16. Since $19 > 16$, further search is restricted to the 13th position and above.

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

$i=9, j=16, m=12, 19 > a_{12}=16, i=m+1=13$

3. 1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

$i=13, j=16, m=14, 19 \geq a_{14}=19, j=14$

4. 1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

$i=13, j=14, m=13, 19 > a_{13}=18, i=m+1=14, i \nless j$ while 循环结束

5. Now the list has a single element and the loop ends. Since $19=19$, the location 14 is returned.

Binary Search₂

Here is a description of the binary search algorithm in pseudocode(伪码).

```
procedure binary search( $x$ : integer,  $a_1, a_2, \dots, a_n$ : increasing integers)
   $i := 1$  { $i$  is the left endpoint of interval}
   $j := n$  { $j$  is right endpoint of interval}
  while  $i < j$ 
     $m := \lfloor (i + j) / 2 \rfloor$ 
    if  $x > a_m$  then  $i := m + 1$ 
    else  $j := m$ 
  if  $x = a_i$  then  $location := i$ 
  else  $location := 0$ 
  return  $location$  {location is the subscript  $i$  of the term  $a_i$  equal to  $x$ ,
    or 0 if  $x$  is not found}
```

Ex3: Worst-Case Complexity of Binary Search(二分搜索)

Example: Describe the time complexity of binary search in terms of the number of comparisons used.

```
procedure binary search( $x$ : integer,  $a_1, a_2, \dots, a_n$ : increasing integers)
   $i := 1$  { $i$  is the left endpoint of interval}
   $j := n$  { $j$  is right endpoint of interval}
  while  $i < j$ 
     $m := \lfloor (i + j) / 2 \rfloor$ 
    if  $x > a_m$  then  $i := m + 1$ 
    else  $j := m$ 
  if  $x = a_i$  then  $location := i$ 
  else  $location := 0$ 
  return  $location$  {location is the subscript  $i$  of the term  $a_i$  equal to  $x$ , or 0 if  $x$  is not found}
```

Solution: Assume (for simplicity) $n = 2^k$ elements. Note that $k = \log n$.

- Two comparisons are made at each stage; $i < j$, and $x > a_m$.
- At the first iteration the size of the list is 2^k and after the first iteration it is 2^{k-1} . Then 2^{k-2} and so on until the size of the list is $2^1 = 2$.
- At the last step, a comparison tells us that the size of the list is the size is $2^0 = 1$ and the element is compared with the single remaining element.
- Hence, at most $2k + 2 = 2 \log n + 2$ comparisons are made.
- Therefore, the time complexity is $\Theta(\log n)$, better than linear search.

Bubble Sort₂

procedure bubblesort($a_1, \dots, a_n$: real numbers
with $n \geq 2$)

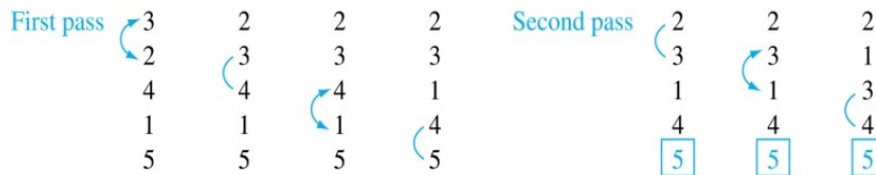
for $i := 1$ to $n - 1$

for $j := 1$ to $n - i$

if $a_i > a_{j+1}$ **then** interchange a_i and a_{j+1}

{ $a_1, \dots, a_n$ is now in increasing order}

Example: Show the steps of bubble sort with 3 2 4 1 5



numbers in color
guaranteed to be in correct order

At the first pass the largest element has been put into the correct position

At the end of the second pass, the 2nd largest element has been put into the correct position.

In each subsequent pass, an additional element is put in the correct position.

[Jump to long description](#)

Ex5: Worst-Case Complexity of Bubble Sort(气泡排序)

Example: What is the worst-case complexity of bubble sort in terms of the number of comparisons made?

```
procedure bubblesort( $a_1, \dots, a_n$ : real numbers  
                    with  $n \geq 2$ )  
  for  $i := 1$  to  $n - 1$   
    for  $j := 1$  to  $n - i$   
      if  $a_j > a_{j+1}$  then interchange  $a_j$  and  $a_{j+1}$   
  { $a_1, \dots, a_n$  is now in increasing order}
```

Solution: A sequence of $n-1$ passes(遍) is made through the list. On each pass $n - i$ comparisons are made.

$$(n-1) + (n-2) + \dots + 2 + 1 = \frac{n(n-1)}{2}$$

The worst-case complexity of bubble sort is $\Theta(n^2)$ since

$$\frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n.$$

Insertion Sort (Optional)₂

Example: Show all the steps of insertion sort with the input: 3 2 4 1 5

- i. 2 3 4 1 5 (*first two positions are interchanged*)
- ii. 2 3 4 1 5 (*third element remains in its position*)
- iii. 1 2 3 4 5 (*fourth is placed at beginning*)
- iv. 1 2 3 4 5 (*fifth element remains in its position*)

Insertion Sort (Optional) ₁

Insertion sort begins with the 2nd element. It compares the 2nd element with the 1st and puts it before the first if it is not larger.

3 2 4 1 5

j=2 i=1 a2 > a1 while 结束 m=a2=2
k=0:0 循环 a2:=a1 a1=m=2

2 3 4 1 5

j=3 i=1 a3 > a1 i=2 a3 > a2 i=3 a3 > a3 while 结束
m=a3 for k=0:-1 不计算 a3=m

2 3 4 1 5

j=4 i=1 a4 > a1 while 结束 m=a4
k=0:(4-1-1) a4=a3 a3=a2 a2=a1
a1=m

1 2 3 4 5

j=5 i=1 a5 > a1 i=2 a5 > a2 i=3 a5 > a3 i=4 a5 > a4
i=5 a5 > a5 while 结束
m=a5 k=0:-1 不循环 a5=m

1 2 3 4 5

procedure *insertion sort*

($a_1, \dots, a_n$: real numbers with $n \geq 2$)

for $j := 2$ to n

$i := 1$

while $a_j > a_i$

$i := i + 1$

$m := a_j$

for $k := 0$ to $j - i - 1$

$a_{j-k} := a_{j-k-1}$

$a_i := m$

{Now $a_1, \dots, a_n$ is in increasing order}

<http://alrightchiu.github.io/SecondRound/comparison-sort-insertion-sortcha-ru-pai-xu-fa.html>

Ex6: Worst-Case Complexity of Insertion Sort (插入排序) (Optional)

Example: What is the worst-case complexity of insertion sort in terms of the number of comparisons made?

Solution: The total number of comparisons are

$$2 + 3 + \cdots + n = \frac{n(n-1)}{2} - 1$$

Therefore the complexity is $\Theta(n^2)$.

```
procedure insertion sort( $a_1, \dots, a_n$ :  
    real numbers with  $n \geq 2$ )  
    for  $j := 2$  to  $n$   
         $i := 1$   
        while  $a_j > a_i$   
             $i := i + 1$   
         $m := a_j$   
        for  $k := 0$  to  $j - i - 1$   
             $a_{j-k} := a_{j-k-1}$   
         $a_i := m$   
    { $a_1, \dots, a_n$  is in increasing order}
```

3.3.5 Understanding the Complexity of Algorithms₁

TABLE 1 Commonly Used Terminology for the Complexity of Algorithms.

<i>Complexity</i>	<i>Terminology</i>	
$\Theta(1)$	Constant complexity	
$\Theta(\log n)$	Logarithmic complexity	Binary search $\Theta(\log n)$
$\Theta(n)$	Linear complexity	Linear search $\Theta(n)$
$\Theta(n \log n)$	Linearithmic(线性对数) complexity	Merge sort
$\Theta(n^b)$ $b \geq 1$	Polynomial complexity	Bubble sort ,Insertion sort $\Theta(n^2)$
$\Theta(b^n)$, where $b > 1$	Exponential complexity	
$\Theta(n!)$	Factorial complexity	

Understanding the Complexity of Algorithms₂

TABLE 2 The Computer Time Used by Algorithms.

<i>Problem Size</i>	<i>Bit Operations Used</i>					
<i>n</i>	<i>log n</i>	<i>n</i>	<i>n log n</i>	<i>n²</i>	<i>2ⁿ</i>	<i>n!</i>
10	3×10^{-11} s	10^{-10} s	3×10^{-10} s	10^{-9} s	10^{-8} s	3×10^{-7} s
10^2	7×10^{-11} s	10^{-9} s	7×10^{-9} s	10^{-7} s	4×10^{11} yr	*
10^3	1.0×10^{-10} s	10^{-8} s	1×10^{-7} s	10^{-5} s	*	*
10^4	1.3×10^{-10} s	10^{-7} s	1×10^{-6} s	10^{-3} s	*	*
10^5	1.7×10^{-10} s	10^{-6} s	2×10^{-5} s	0.1 s	*	*
10^6	2×10^{-10} s	10^{-5} s	2×10^{-4} s	0.17 min	*	*

Assuming that each bit operation takes 10^{-11} seconds
 Times of more than 10^{100} years are indicated with an *.

$$20! = 2,432,902,008,176,640,000.$$

3.3 Complexity of Algorithms Section Summary

Time Complexity

Worst-Case Complexity

Understanding the Complexity of Algorithms

Chapter Summary

Algorithms

Growth of Functions

- Big- O and other Notation

Complexity of Algorithms

2. Give a big- O estimate for the number additions used in this segment of an algorithm.

$t := 0$

for $i := 1$ **to** n

for $j := 1$ **to** n

$t := t + i + j$

4. Give a big- O estimate for the number of operations, where an operation is an addition or a multiplication, used in this segment of an algorithm (ignoring comparisons used to test the conditions in the **while** loop).

```
 $i := 1$   
 $t := 0$   
while  $i \leq n$   
     $t := t + i$   
     $i := 2i$ 
```

4. The value of i keeps doubling, so the loop terminates after k iterations as soon as $2^k > n$. The value of k that makes this happen is $O(\log n)$, because $2^{\log n} = n$. Within the loop there are two additions or multiplications, so the answer to the question is $O(\log n)$.

Homework

Page 229: 7ab., 25ab, 34a.

Page 241: 1, 3.

8. Find the least integer n such that $f(x)$ is $O(x^n)$ for each of these functions.

a) $f(x) = 2x^2 + x^3 \log x$

b) $f(x) = 3x^5 + (\log x)^4$

16. Describe an algorithm for finding the smallest integer in a finite sequence of natural numbers.

16. We let min be the smallest element found so far. At the end, it is the smallest element, since we update it as necessary as we scan through the list.

```
procedure smallest( $a_1, a_2, \dots, a_n$  : natural numbers)
   $min := a_1$ 
  for  $i := 2$  to  $n$ 
    if  $a_i < min$  then  $min := a_i$ 
  return  $min$  {the smallest integer among the input }
```

5. How many comparisons are used by the algorithm given in Exercise 16 of Section 3.1 to find the smallest natural number in a sequence of n natural numbers?

5. $2n - 1$

3. Give a big- O estimate for the number of operations, where an operation is a comparison or a multiplication, used in this segment of an algorithm (ignoring comparisons used to test the conditions in the **for** loops, where $a_1, a_2, \dots, a_n$ are positive real numbers).

$m := 0$

for $i := 1$ **to** n

for $j := i + 1$ **to** n

$m := \max(a_i a_j, m)$

$O(n^2)$

18. How much time does an algorithm take to solve a problem of size n if this algorithm uses $2n^2 + 2^n$ operations, each requiring 10^{-9} seconds, with these values of n ?

a) 10 **b)** 20

18. We are asked to compute $(2n^2 + 2^n) \cdot 10^{-9}$ for each of these values of n . When appropriate, we change the units from seconds to some larger unit of time.

a) 1.224×10^{-6} seconds **b)** approximately 1.05×10^{-3} seconds